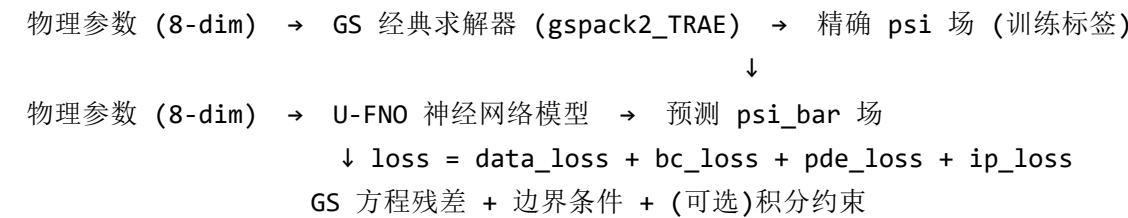


GS_PINO 技术文档

1. 项目概述

GS_PINO 是一个基于**神经网络算子**的固定边界 Grad-Shafranov (GS) 方程求解器。项目使用 **U-FNO** (U-Net + Fourier Neural Operator) 架构，学习从 8 个物理参数到归一化极向磁通 `psi_bar` 的映射。

核心流程



文件结构

src/gs_pino/	
├─ __init__.py	# 包初始化
├─ geometry.py	# LCFS 几何工具: Miller 参数化、掩码、SDF
├─ solvers.py	# GS 求解器适配器: 解析备用 + gspack2_TRAE 封装
├─ generate_dataset.py	# 数据集生成 CLI: 采样参数、调用求解器、保存 .npz
├─ data.py	# 数据集类 GSDataset: 加载、归一化、输入通道构建
├─ models.py	# U-FNO 模型定义: SpectralConv2d, UFNOblock, UFN02d
├─ losses.py	# 损失函数: masked MSE, GS 残差, Ip/betap 约束
├─ train.py	# 训练 CLI: 训练循环、验证、checkpoint 保存
└─ evaluate.py	# 评估 CLI: 指标计算、可视化对比图

2. 物理背景: Grad-Shafranov 方程

托卡马克等离子体平衡由 Grad-Shafranov 方程描述:

$$\Delta^*\psi + \mu_0 R^2 p'(\psi) + FF'(\psi) = 0$$

其中：

- ψ — 极向磁通函数 (Poloidal magnetic flux)
- Δ^* — 修正 Laplace 算子: $\Delta^*\psi = \partial^2\psi/\partial R^2 - (1/R)\cdot\partial\psi/\partial R + \partial^2\psi/\partial Z^2$
- $p(\psi)$ — 等离子体压强剖面
- $F(\psi) = R\cdot B_\phi$ — 极向电流函数
- $\mu_0 = 4\pi\times 10^{-7}$ — 真空磁导率

剖面模型 (Jeon 2015)

使用 `ConstrainBetapIp` 参数化电流和压强剖面：

$$p'(\psi_N) = L \cdot \text{Beta0} / R_{\text{axis}} \cdot (1 - \psi_N^{\{\alpha m\}})^{\{\alpha n\}}$$

$$FF'(\psi_N) = \mu_0 \cdot L \cdot (1 - \text{Beta0}) \cdot R_{\text{axis}} \cdot (1 - \psi_N^{\{\alpha m\}})^{\{\alpha n\}}$$

其中 $\psi_N = (\psi - \psi_{\text{lcfs}}) / (\psi_{\text{axis}} - \psi_{\text{lcfs}})$ 是归一化磁通，LCFS 处为 0，磁轴处为 1。

3. 数据生成流程

3.1 参数采样 (`generate_dataset.py::sample_params`)

```
ranges = {  
    "R0":      (0.8, 1.5),      # 主半径 [m]  
    "a":       (0.3, 0.7),      # 小半径 [m]  
    "kappa":   (1.0, 2.0),      # 拉长比  
    "delta":    (0.0, 0.5),      # 三角形变  
    "Ip":      (1e5, 5e5),      # 等离子体电流 [A]  
    "betap":    (0.3, 1.5),      # 极向比压  
    "alpha_m": (0.5, 3.0),      # 电流剖面指数 1  
    "alpha_n": (0.5, 3.0),      # 电流剖面指数 2  
}
```

3.2 经典求解器调用 (`solvers.py::GSSolverAdapter`)

经典求解器来自 [GS_solver](#) (本项目中以 `gspack2_TRAE` 目录引用)，使用有限差分法 + Picard 迭代求解固定边界 GS 方程。

```
物理参数 (dict)
↓
FixedBoundaryEquilibrium(R0, a, kappa, delta, fix_bndry_zero=True, nx=nr, ny=nz)
↓ + ConstrainBetapIp(betap, Ip, alpha_m, alpha_n, Raxis=R0)
↓
picard.solve(eq, pro, maxits=50, rtol=1e-5, anderson_m=5)
↓
返回: {R, Z, psi, psi_bar, psi_lcfs=0, psi_axis, R_axis, Z_axis,
       plasma_mask, profile_params=[L, Beta0]}
```

fix_bndry_zero=True 使求解器直接返回 LCFS 处 $\psi=0$ 的结果，省去后续平移步骤，同时求解速度提升约 1.6 倍。

3.3 几何通道 (geometry.py)

每个样本在保存时包含 4 个几何通道：

通道	含义	计算方式
mask	LCFS 内部二值掩码	求解器返回 plasma_mask 或 $\rho \leq 1.0$
sdf	符号距离函数（近似）	$\rho - 1.0$ ，内部为负
rho	归一化磁面半径	Miller 近似
theta	极向角	$\text{atan2}(Z/(\kappa a), (R-R_0)/a)$

3.4 保存格式 (.npz)

```
data/gs_large.npz
├─ R, Z           [n, nr, nz]   - 计算网格坐标
├─ params         [n, 8]       - 8 个物理参数
├─ psi            [n, nr, nz]   - 真实 psi (LCFS=0)
├─ psi_bar        [n, nr, nz]   - 归一化 psi (LCFS=0, axis=1)
├─ mask           [n, nr, nz]   - 等离子体掩码
├─ sdf            [n, nr, nz]   - 符号距离函数
├─ rho            [n, nr, nz]   - 归一化半径
├─ theta          [n, nr, nz]   - 极向角
├─ axes           [n, 4]       - [R_axis, Z_axis, psi_lcfs, psi_axis]
├─ profile_params [n, 2]       - [L, Beta0]
└─ param_names    [8]         - 参数名称
```

4. U-FNO 模型架构

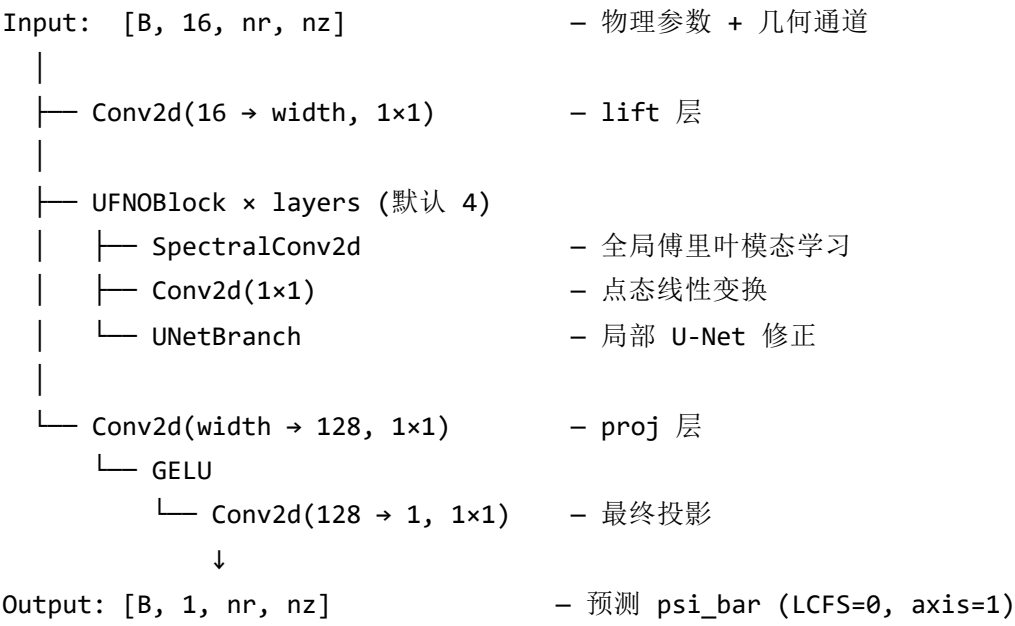
4.1 输入通道构建 (data.py::build_input)

输入张量形状: [C, nr, nz] , 共 **8 + 8 = 16 通道**:

- 8 个几何/坐标通道:
[(R-R0)/a, Z/a, Z/(kappa*a), mask, sdf, rho, sin(theta), cos(theta)]
- 8 个物理参数通道 (归一化后广播到全网格):
[R0_norm, a_norm, kappa_norm, delta_norm, Ip_norm, betap_norm, alpha_m_norm, alpha_n_norm]

归一化使用训练集的均值和标准差: $p_norm = (p - mean) / std$ 。

4.2 整体架构



4.3 各模块详细结构

4.3.1 SpectralConv2d (傅里叶谱卷积)

```
Input: [B, C_in, H, W]
|
├─ torch.fft.rfft2                - 实 FFT, 输出 [B, C_in, H, W/2+1]
|
├─ 低频模态加权:
|   ├─ 正频率: out_ft[:, :, :m1, :m2] = weights1 * x_ft[:, :, :m1, :m2]
|   └─ 负频率: out_ft[:, :, -m1:, :m2] = weights2 * x_ft[:, :, -m1:, :m2]
|   weights shape: [C_in, C_out, modes1, modes2] (复数)
|   einsum: "bixy,ioxy->boxy"
|
└─ torch.fft.irfft2                - 逆 FFT, 输出 [B, C_out, H, W]
```

关键参数:

- modes1 : R 方向保留的傅里叶模态数 (默认 32)
- modes2 : Z 方向保留的傅里叶模态数 (默认 32)
- 模态数会自动裁剪为不超过网格尺寸的一半

4.3.2 UNetBranch (局部修正分支)

```
Input: [B, width, H, W]
|
├─ AvgPool2d(2x2, ceil_mode)      - 下采样, [B, width, H/2, W/2]
|
├─ Conv2d(width→width, 3x3, pad=1) + GELU
|   └─ Conv2d(width→width, 3x3, pad=1) + GELU
|
├─ interpolate(bilinear, H, W)    - 上采样恢复原始分辨率
|
└─ Conv2d(2*width→width, 3x3, pad=1) + GELU - 拼接跳跃连接后
    └─ Conv2d(width→width, 3x3, pad=1)      - 最终输出
```

4.3.3 UFNOBlock (完整块)

Output = GELU(SpectralConv2d(x) + PointwiseConv2d(x) + UNetBranch(x))

三个分支相加后经过 GELU 激活:

- **Spectral path**: 全局傅里叶模态学习
- **Pointwise path**: 逐点通道混合 (1×1 卷积)
- **UNet path**: 局部特征提取和下采样/上采样融合

4.4 完整张量形状流程示例

以 batch=16, nr=129, nz=129, width=64, layers=4, modes1=32, modes2=32 为例：

层	输出形状
Input	[16, 16, 129, 129]
lift (Conv2d 1×1)	[16, 64, 129, 129]
UFNOBlock 1	[16, 64, 129, 129]
└─ SpectralConv2d	[16, 64, 129, 129]
└─ rfft2	[16, 64, 129, 65]
└─ modal multiply	[16, 64, 129, 65]
└─ irfft2	[16, 64, 129, 129]
└─ Conv2d(1×1)	[16, 64, 129, 129]
└─ UNetBranch	[16, 64, 129, 129]
└─ AvgPool2d	[16, 64, 65, 65] (ceil_mode)
└─ Conv2d(3×3)×2	[16, 64, 65, 65]
└─ interpolate	[16, 64, 129, 129]
└─ Conv2d(3×3)×2	[16, 64, 129, 129]
UFNOBlock 2-4 (同结构)	[16, 64, 129, 129]
proj (Conv2d 1×1→128→1)	[16, 1, 129, 129]

5. 损失函数 (losses.py)

5.1 数据损失: masked_mse

仅在 LCFS 内部区域计算均方误差：

$$\text{loss_data} = \frac{\sum[(\text{pred} - \text{target})^2 \cdot \text{mask}]}{\sum[\text{mask}]}$$

5.2 边界条件损失: boundary_band_loss

在 LCFS 附近 (|sdf| < 0.04) 惩罚非零值，强制边界条件 $\psi_{\text{bar}}|_{\text{LCFS}} = 0$ ：

```
band = (|sdf| < width).float()          # width = 0.04
loss_bc =  $\Sigma[\text{pred}^2 \cdot \text{band}] / \Sigma[\text{band}]$ 
```

5.3 GS 方程残差: gs_residual_loss

计算 $\Delta\psi_{\text{bar}}$ 并与源项比较:

步骤 1: 计算 $\Delta\psi_{\text{bar}}$ (有限差分)

```
 $\partial^2\psi/\partial R^2 \approx (\psi[i+1,j] - 2*\psi[i,j] + \psi[i-1,j]) / dR^2$ 
 $\partial\psi/\partial R \approx (\psi[i+1,j] - \psi[i-1,j]) / (2*dR)$ 
 $\partial^2\psi/\partial Z^2 \approx (\psi[i,j+1] - 2*\psi[i,j] + \psi[i,j-1]) / dZ^2$ 

 $\Delta\psi_{\text{bar}} = \partial^2\psi/\partial R^2 - (1/R) \cdot \partial\psi/\partial R + \partial^2\psi/\partial Z^2$ 
```

步骤 2: 计算源项

```
psiN = 1.0 - psi_bar          # 归一化磁通 [0,1]
shape = (1 - psiN{αm}){αn}    # 剖面形状
S = μ0 · L · [Beta0·R2/R0 + (1-Beta0)·R0] · shape
```

步骤 3: 残差

```
dpsi = psi_axis - psi_lcfs    # 归一化因子
residual = lap_psi_bar + S / dpsi
loss_pde =  $\Sigma[\text{residual}^2 \cdot \text{mask}] / \Sigma[\text{mask}]$ 
```

5.4 Ip 约束损失 (可选): ip_constraint_loss

通过积分 J_ϕ 计算预测 I_p 并与目标比较:

```
 $J_\phi = L \cdot [\text{Beta0} \cdot R/R0 + (1-\text{Beta0}) \cdot R0/R] \cdot \text{shape}$ 
Ip_pred =  $\iint J_\phi \, dR \, dZ$ 
loss_ip =  $(I_{p\_pred} - I_{p\_target})^2 / I_{p\_target}^2$ 
```

5.5 betap 约束损失 (可选): betap_constraint_loss

$$\beta p = 2\mu_0 \cdot \iint p \cdot R \, dR \, dZ \, / \, \iint B_{\text{pol}}^2 \cdot R \, dR \, dZ$$

$$p(\psi_N) = \text{dpsi} \cdot L \cdot \text{Beta0}/R0 \cdot \int_0^{1-\psi_N} (1-s^{\{\alpha_m\}})^{\{\alpha_n\}} \, ds \quad \# \text{ 数值积分}$$

$$B_{\text{pol}}^2 = B_r^2 + B_z^2$$

$$B_r = -(1/R) \cdot \partial\psi / \partial Z$$

$$B_z = (1/R) \cdot \partial\psi / \partial R$$

5.6 总损失

$$\begin{aligned} \text{loss} = & \text{loss_data} + \text{bc_weight} * \text{loss_bc} + \text{pde_weight} * \text{loss_pde} \\ & + \text{ip_weight} * \text{loss_ip} + \text{betap_weight} * \text{loss_betap} \end{aligned}$$

默认权重: bc=0.05, pde=0.01, ip=0.0, betap=0.0

6. 训练流程 (train.py)

6.1 数据划分

总样本 n

- └─ 训练集: n * 0.70 (随机排列后取前 70%)
- └─ 验证集: n * 0.15 (中间 15%)
- └─ 测试集: n * 0.15 (最后 15%)

6.2 每 epoch 步骤

```
for x, y, mask, sdf, params, meta_list in loader:
    meta = stack_metadata(meta_list)      # 合并元数据为 batch tensor

    pred = model(x)

    loss_data = masked_mse(pred, y, mask)
    loss_bc   = bc_weight * boundary_band_loss(pred, sdf)
    loss_pde  = pde_weight * gs_residual_loss(pred, ..., meta)
    loss_ip   = ip_weight * ip_constraint_loss(pred, ..., meta)

    loss = loss_data + loss_bc + loss_pde + loss_ip

    if training:
        loss.backward()
        optimizer.step()
```

6.3 Checkpoint 保存

当验证损失最低时保存 best.pt :

```
torch.save({
    "model": model.state_dict(),
    "args": vars(args),           # 命令行参数
    "param_mean": train_ds.param_norm.mean,
    "param_std": train_ds.param_norm.std,
    "test_indices": test_idx,     # 测试集索引
}, "best.pt")
```

6.4 训练监控

每个 epoch 记录到 history.json :

- train_data, train_bc, train_pde, train_ip, train_total
- val_data, val_bc, val_pde, val_ip, val_total

自动生成 training_curves.png 展示各损失变化曲线。

7. 评估流程 (evaluate.py)

7.1 指标

```
{
    "masked_mse": 0.00014,           // 测试集 masked MSE
    "relative_l2_mean": 0.023,       // 平均相对 L2 误差
    "relative_l2_median": 0.016,     // 中位数
    "relative_l2_p95": 0.065,       // 95 分位数
    "relative_l2_max": 0.19          // 最大值
}
```

7.2 可视化

每个测试样本生成三面板对比图：

- 1. **true psi** — 经典求解器输出的真实 psi 场
- 2. **pred psi** — 模型预测的 psi 场 (psi_bar 还原为真实 psi)
- 3. **pred - true** — 预测误差

整体统计图：

- summary_error_histogram.png — 误差分布直方图
- summary_error_vs_parameters.png — 误差与各输入参数的关系散点图

8. 配置文件与命令行参数

8.1 generate_dataset

参数	默认值	说明
--out	data/gs_fixed_boundary.npz	输出路径
--n-samples	64	样本数
--nr	64	R 网格数
--nz	64	Z 网格数

参数	默认值	说明
--seed	42	随机种子
--rtol	1e-5	求解器收敛容差

8.2 train

参数	默认值	说明
--data	data/gs_fixed_boundary.npz	数据集路径
--output-dir	outputs/run	输出目录
--epochs	20	训练轮数
--batch-size	4	批次大小
--lr	1e-3	学习率
--width	32	模型通道宽度
--modes1	16	R 方向傅里叶模态数
--modes2	16	Z 方向傅里叶模态数
--layers	4	U-FNO 块数量
--pde-weight	0.01	PDE 残差损失权重
--bc-weight	0.05	边界损失权重
--ip-weight	0.0	lp 约束损失权重
--seed	0	随机种子

8.3 evaluate

参数	默认值	说明
--data	data/gs_fixed_boundary.npz	数据集路径
--checkpoint	必填	模型权重文件
--output-dir	outputs/eval	输出目录

参数	默认值	说明
--batch-size	4	评估批次大小
--max-plots	12	最大可视化图数

9. 运行示例

9.1 端到端流程

```
# 1. 生成数据集（129×129 网格，1000 样本）
python -m gs_pino.generate_dataset --out data/gs_large.npz \
    --n-samples 1000 --nr 129 --nz 129 --seed 42 --rtol 1e-5

# 2. 训练模型
python -m gs_pino.train --data data/gs_large.npz \
    --epochs 200 --batch-size 16 --width 64 --modes1 32 --modes2 32 \
    --layers 4 --pde-weight 0.01 \
    --output-dir outputs/large

# 3. 评估模型
python -m gs_pino.evaluate --data data/gs_large.npz \
    --checkpoint outputs/large/best.pt \
    --output-dir outputs/large_eval --max-plots 10
```

9.2 快速测试

```
# 小数据集快速验证
python -m gs_pino.generate_dataset --out data/gs_smoke.npz \
    --n-samples 20 --nr 65 --nz 65 --seed 42

python -m gs_pino.train --data data/gs_smoke.npz \
    --epochs 10 --batch-size 4 --width 16 --modes1 8 --modes2 8 \
    --layers 2 --pde-weight 0.01 --output-dir outputs/smoke

python -m gs_pino.evaluate --data data/gs_smoke.npz \
    --checkpoint outputs/smoke/best.pt \
    --output-dir outputs/smoke_eval --max-plots 3
```

10. 参考结果

Large 配置（984 训练样本，129×129 网格，200 epochs）

指标	值
最佳验证损失	0.00085
测试 masked MSE	0.00014
相对 L2 均值	2.3%
相对 L2 中位数	1.6%
相对 L2 P95	6.5%
相对 L2 最大值	19.3%
训练时间	~46 min (CUDA)
数据集生成时间	~55 min (CPU)

11. 依赖

Python 包

- torch >= 2.0
- numpy
- matplotlib
- tqdm

外部求解器 (可选)

- [GS_solver](#) — 经典有限差分 GS 求解器，需克隆至与 GS_PINO 同级目录（默认目录名为 gspack2_TRAE ）